

⑧ Assignment - 1

DSUC

* **Data structure** :- In computer science, DS is a data organization, & storage format that is usually chosen for efficient access to data.

↓
DS
Primitive (one data element)
(int, float, char etc)
, pointer

↓
Non-primitive (multiple data elements)
↓
Linear (continuous memory)
↓
Non-linear (non-continuous)

Example data type → Static
(10, 20, 30, 40, 50)
↓
Array

Dynamic (different data representation)
↓
Linked list, stack, queue

* **Operation in DS** :-

- 1) searching
- 2) sorting
- 3) Insertion
- 4) Deletion
- 5) updation
- 6) Traversing

* **Algorithm** :- sequence of steps to solve a problem.

* **Data Base** :- Collection of information in permanent storage for faster retrieval and updation.

⑨ Array operations :-

* **Array searching** ← Linear binary

→ **Linear searching**

Ex :- #include <stdio.h>

int main() {

int num;

printf("Enter number : ");

scanf("%d", &num);

int arr[5] = {1, 2, 3, 4, 5};

int flag = 0;

for (int i = 0; i < 5; i++) {

if (arr[i] == num) {

printf("Element found at arr[%d]", i);

flag = 1; }

}

if (flag == 0) {

printf("Element not found");

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

}

Assignment - 2

→ **Binary searching** :-

Ex :-

#include <stdio.h>

int main() {

int num;

int arr[10] = {5, 7, 11, 15, 19, 28, 30, 39, 42, 45};

int lb = 0, ub = 9, flag = 0, mid;

printf("Enter searching number : ");

scanf("%d", &num);

while (lb <= ub) {

mid = (lb + ub) / 2;

if (arr[mid] == num) {

flag = 1; }

if (arr[mid] < num) {

lb = mid + 1; }

if (arr[mid] > num) {

ub = mid - 1; }

}

if (flag == 1) {

printf("Element found at arr[%d]", mid);

}

if (flag == 0) {

printf("Element not found");

}

}

}

}

}

}

}

}

}

}

}

}

```

if (mid == num) {
    printf("Item found at arr[%d]", mid);
    flag++;
    break;
}
if (mid < num) {
    lb = mid + 1;
}
if (mid > num) {
    ub = mid - 1;
}
if (flag == 0) {
    printf("Item not found");
}
}

```

Assignment - 3

* Insertion :-

```

#include <stdio.h>
int main() {
    int arr[10] = {1, 3, 5, 9, 11, 13, 17, 21, 22};
    printf("Before insertion :");
    for (int i = 0; i < 9; i++) {
        printf("%d\t", arr[i]);
    }
    int index, value;
    printf("Enter index where the value to be insert");
    scanf("%d", &index);
    printf("Enter value to be insert");
    scanf("%d", &value);
    for (int i = 8; i >= index; i--) {

```

```

arr[i+1] = arr[i];
arr[index] = value;
printf("Updated array after insertion");
for (int i = 0; i < 10; i++) {
    printf("%d\t", arr[i]);
}
}

```

Assignment - 4

* Deletion :-

```

#include <stdio.h>
int main() {
    int arr[10] = {1, 3, 5, 7, 9, 11, 13, 15, 17, 19};
    int num, flag = 0, index;
    printf("Enter number to delete");
    scanf("%d", &num);
    printf("Original array");
    for (int i = 0; i < 10; i++) {
        printf("%d\t", arr[i]);
    }
    for (int i = 0; i < 10; i++) {
        if (arr[i] == num) {
            printf("Item found at arr[%d]", i);
            flag++;
            index = i;
        }
    }
    if (flag == 0) {
        printf("No value found");
    }
}

```



```

for (int i = index; i < 10; i++) {
    arr[i] = arr[i+1];
}
printf("After deletion of array");
for (int i = 0; i < 9; i++) {
    printf("%d ", arr[i]);
}

```

Assignment - 5
 * Array Addressing :-
 [11 12 13 14
 21 22 23 24
 31 32 33 34] $m \times n$
 3×4

Suppose we have to find address of
 23 :-
 address of 23 $\Rightarrow$

According to row major :-

$((i \times n) + j) \times \text{size} + \text{Base address}$
 array starts from 1 :-
 $((i-1) \times n + (j-1) \times \text{size}) + \text{Base address}$

According to column major :-

$((j \times m) + i) \times \text{size} + \text{Base address}$
 $((j-1) \times m + (i-1) \times \text{size}) + \text{Base address}$

(12 - array)

[0 1 2 3 4]
 3000 3001 3002 3003 3004

$\Rightarrow i \times \text{size} + \text{Base address}$

Assignment - 6
 * Linked List :-
 It is collection of 'n' number of data which is
 implemented using the concept of dynamic memory
 allocation. It stores as many elements you want
 Memory creates during run-time.

$\rightarrow$ Program :-

```

#include <stdio.h>
#include <stdlib.h>
struct list {
    int data;
    struct list * next;
};
int main() {
    struct list * head = NULL;
    typedef struct list node;

```

```

    int main() {
        int choice;
        while (1) {
            printf("1 -> Create list \n 2 -> Display list \n 3 -> Exit \n Enter your choice");
            scanf("%d", &choice);

```

```

            if (choice == 1) {
                choice create();
            }

```

(14)

```

else if (choice == 2) {
    display();
}
else if (choice == 3) {
    break;
}
else {
    printf("Wrong choice");
}
return 0;
}

```

```

void create_choice() {
    node *tmp, *tmp1;
    tmp = (node *) malloc(sizeof(node));
    tmp->next = NULL;
    printf("Enter value of node");
    scanf("%d", tmp->data);
    if (start == NULL) {
        start = tmp;
    }
    else {
        tmp1 = start;
        while (tmp1->next != NULL) {
            tmp1 = tmp1->next;
        }
        tmp1->next = tmp;
    }
}

```

```

void display() {
    node *tmp;
    tmp = start;
    if (tmp == NULL) {
        printf("Empty");
    }
    else

```

(15)

```

{
    while (tmp1 != NULL) {
        printf("%d", tmp->data);
        tmp = tmp->next;
    }
}
}

```

→ Insert program at the beginning of the program

```

void insert() {
    node *tmp;
    tmp = (node *) malloc(sizeof(node));
    printf("Enter data of new node");
    scanf("%d", tmp->data);
    tmp->next = start;
    start = tmp;
    printf("Node is successfully added");
}

```

→ Insert program at the end of list

```

void insert() {
    node *tmp, *tmp1;
    tmp = (node *) malloc(sizeof(node));
    printf("Enter data of new node");
    scanf("%d", tmp->data);
    while (tmp1 = start; tmp->next = NULL;
    while (tmp1->next != NULL) {
        tmp1 = tmp1->next;
    }
    tmp1->next = tmp;
}

```


16

→ Program to count nodes of linked list :-

```
void count() {
    node * tmp;
    int count = 0;
    tmp = start;
    while (tmp != NULL) {
        count++;
        tmp = tmp->next;
    }
    printf("No. of nodes = %d", count);
}
```

→ Program to insert at any position :-

```
void anyinsert() {
    int pos, count = 0;
    node * tmp, * tmp1;
    tmp = start;
    while (tmp != NULL) {
        tmp1 = tmp->next;
        count++;
    }
    printf("Enter position to insert");
    scanf("%d", &pos);
    if (pos < 1 || pos > count + 1) {
        printf("Invalid position");
    }
    else {
        tmp = (node*) malloc (size of (node));
        printf("Enter data of new node");
        scanf("%d", tmp->data);
    }
}
```

17

```
if (pos == 1) {
    tmp1->next = start;
    start = tmp1;
}
else {
    tmp1 = start;
    while (tmp1) {
        for (int i = 1; i < pos - 1; i++) {
            tmp1 = tmp1->next;
        }
        tmp1->next = tmp->next;
        tmp->next = tmp1;
    }
}
```

→ Program to delete first element :-

```
void delete() {
    node * tmp;
    tmp = start;
    if (tmp == NULL) {
        printf("Empty linked list");
    }
    else {
        start = tmp->next;
        free(tmp);
        printf("First node deleted with value %d", tmp->data);
    }
}
```

→ Program to delete last

```
void edelete() {
    node * tmp, * tmp1;
    tmp = start;
    if (tmp->next->next != NULL) {
        tmp1 = tmp->next;
    }
}
```

```

tmp1 = tmp -> next;
tmp -> next = NULL;
free(tmp1);
}

```

→ program to delete any node :-

```

void anydelete() {
    int pos, count = 0;
    node *tmp, *tmp1;
    tmp = start;
    while (tmp != NULL) {
        count++;
        tmp = tmp -> next;
    }
    printf("Enter position to delete:");
    scanf("%d", &pos);
    if (pos < 1 || pos > count)
        printf("Invalid position");
}

```

else

```

if (pos == 1) {
    tmp = start;
    start = tmp -> next;
    free(tmp);
}

```

```

else { tmp = start;
    for (i = 1; i < pos - 1; i++) {
        tmp = tmp -> next;
    }
    tmp1 = tmp -> next;
    tmp -> next = tmp1 -> next;
    free(tmp1);
} } }

```

→ Program to reverse a linked list :-

```

void reverse() {
    struct list * prevnode, * currentnode, * nextnode;
    prevnode = NULL;
    currentnode = headnode = start;
    while (currentnode != NULL) {
        nextnode = currentnode -> next;
        currentnode -> next = prevnode;
        prevnode = currentnode;
        currentnode = nextnode;
    }
    start = prevnode;
}

```

~~*** Doubly linked list ***~~

→ Program to delete by value :-

```

void deletebyvalue() {
    int pos = 1, key; flag = 0;
    struct list * tmp, * tmp1;
    tmp = start;
    printf("Enter item to delete:");
    scanf("%d", &key);
    while (tmp != NULL) {
        if (tmp -> data == key) {
            flag = 1;
            break;
        }
        tmp = tmp -> next;
        pos++;
    }
    if (flag == 0)

```



```

    printf("Stack not found");
}
if (top == -1) {
    printf("Stack is empty");
    return;
}
printf("Element at top position is deleted");
top--;
}
}

```

Assignment 7

* Ques :-

→ Program to implement stack using array?

Push() → Insertion

Pop() → Deletion

Program

```

int arr[5];
int top = -1;

```

```

void push() {
    int n;
    printf("Enter data");
    scanf("%d", &n);
    if (top == -1) {
        printf("overflow");
    }
    else {
        top++;
        arr[top] = n;
    }
}

```

```

pop() {
    if (top == -1) {
        printf("Underflow");
    }
    else {
        top--;
    }
}

```

```

display() {
    for (int i = top; i >= 0; i--) {
        printf("%d", arr[i]);
    }
}

```

→ Stack using linked list :-

```

struct node {
    int data;
    struct node * next;
};
struct node * top = NULL;

```

```

void push() {
    struct node * tmp;
    tmp = (struct node *) malloc (sizeof(struct node));
    printf("Enter data");
    scanf("%d", &tmp->data);
    tmp->next = top;
    top = tmp;
}

```

```

void pop() {
    struct node * tmp;
    tmp = top;
}

```

22

```
top = top -> next;
free(tmp);
}
```

```
void Display() {
    struct node *tmp;
    while (tmp != NULL) {
        printf("%d", tmp->data);
        tmp = tmp->next;
    }
}
```

* Infix $\rightarrow$ <operand> <operator> <operand>
ex $\rightarrow 5 + 1$

* Prefix $\rightarrow$ <operator> <operand> <operand>
ex $\rightarrow + 5 1$
right to left scan

* Postfix $\rightarrow$ <operand> <operand> <operator>
ex $\rightarrow 5 1 +$
left to right scan

(3.9) Video

Date
Page No.

23

Assignment - 8

* Infix to Postfix conversion :-

$(K + L - M * N + (O \wedge P) * W / U / V * T + Q)$

Input	Stack	Expression
(	(	K
K	(	K
+	(+	KL
L	(+	KL +
-	(-	KL + M
M	(-	KL + M
*	(- *	KL + MN
N	(- *	KL + MN * -
+	(+	KL + MN * -
(	(+ (	KL + MN * - O
O	(+ (	KL + MN * - O
^	(+ ^	KL + MN * - O P
P	(+ ^	KL + MN * - O P
)	(+	KL + MN * - O P ^
*	(+ *	KL + MN * - O P ^
W	(+ *	KL + MN * - O P ^ W
/	(+ /	KL + MN * - O P ^ W *
U	(+ /	KL + MN * - O P ^ W * U
/	(+ /	KL + MN * - O P ^ W * U /
V	(+ /	KL + MN * - O P ^ W * U / V
*	(+ *	KL + MN * - O P ^ W * U / V *
T	(+ *	KL + MN * - O P ^ W * U / V * T
+	(+	KL + MN * - O P ^ W * U / V * T +
Q	(+	KL + MN * - O P ^ W * U / V * T + Q
)	-	KL + MN * - O P ^ W * U / V * T + Q +

$\Rightarrow KL + MN * - O P ^ W * U / V * T + Q +$

Assignment - 9

Franchise Info

1

Expression

27

1

57

十

✱

...

七

7

...

11

未刊

17

11

115

1

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	100	101	102	103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127	128	129	130	131	132	133	134	135	136	137	138	139	140	141	142	143	144	145	146	147	148	149	150	151	152	153	154	155	156	157	158	159	160	161	162	163	164	165	166	167	168	169	170	171	172	173	174	175	176	177	178	179	180	181	182	183	184	185	186	187	188	189	190	191	192	193	194	195	196	197	198	199	200	201	202	203	204	205	206	207	208	209	210	211	212	213	214	215	216	217	218	219	220	221	222	223	224	225	226	227	228	229	230	231	232	233	234	235	236	237	238	239	240	241	242	243	244	245	246	247	248	249	250	251	252	253	254	255	256	257	258	259	260	261	262	263	264	265	266	267	268	269	270	271	272	273	274	275	276	277	278	279	280	281	282	283	284	285	286	287	288	289	290	291	292	293	294	295	296	297	298	299	300	301	302	303	304	305	306	307	308	309	310	311	312	313	314	315	316	317	318	319	320	321	322	323	324	325	326	327	328	329	330	331	332	333	334	335	336	337	338	339	340	341	342	343	344	345	346	347	348	349	350	351	352	353	354	355	356	357	358	359	360	361	362	363	364	365	366	367	368	369	370	371	372	373	374	375	376	377	378	379	380	381	382	383	384	385	386	387	388	389	390	391	392	393	394	395	396	397	398	399	400	401	402	403	404	405	406	407	408	409	410	411	412	413	414	415	416	417	418	419	420	421	422	423	424	425	426	427	428	429	430	431	432	433	434	435	436	437	438	439	440	441	442	443	444	445	446	447	448	449	450	451	452	453	454	455	456	457	458	459	460	461	462	463	464	465	466
---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

1

10/27

7

5

4

1

425

之

A

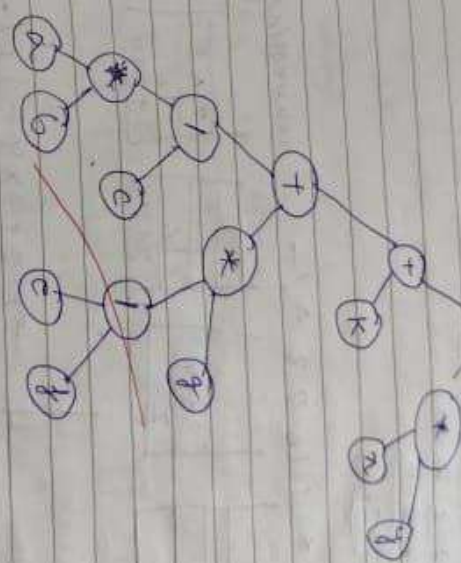
1

51

0.70

1

(22)



Assignment - 10

78

* Inorder : left root right

Preorder : root left right

Postorder : left right root

Types of tree :-

1) Full Binary tree

Each node have must 2 children

2) Complete Binary tree

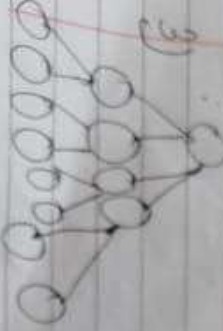
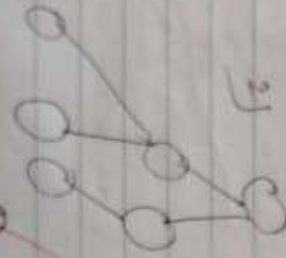
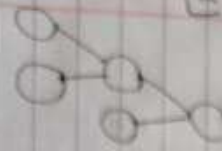
At last level the rightmost child, we have to fill from its test left all have two children

3) Perfect Binary tree

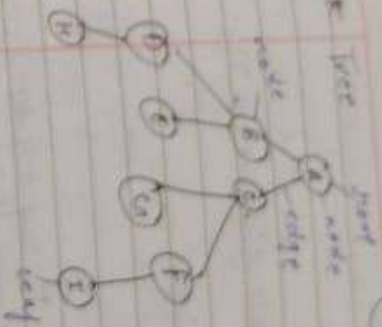
At last level all have internal node has 2 children, and leaves are at same level.

4) Degenerate Binary tree :-

All node must have only 1 children.



* Tree



→ Height of tree on root = 5

→ Depth of H = 5

→ Depth of D = 2

→ leaf are with no child node

* Binary tree :- In a binary tree there is no more than 2 child of a single child.

Assignment - 11

Binary tree by linked list :-

struct tree {

int data;

struct tree * left;

struct tree * right;

};

struct tree * root = NULL;

int main() {

1) create(); root = create();

2) display();

3) exit();

}

L to R (26)

* Evaluating position expression :-

$$234 * + 1623 \wedge / -$$

$$212 + 2 -$$

$$142 -$$

$$12 \rightarrow$$

* $234 * + 1623 \wedge / -$

converting into rpn



$$\Rightarrow 2+3 \times 4-16 / 2 \wedge 3$$

* $2+3 \times 4-16 / 2 \wedge 3$ converting to prefix

$$2 \wedge 3$$

$$2 \wedge 3 2$$

$$2 \wedge 3 2 \wedge$$

$$16 /$$

$$2 \wedge 16 /$$

$$2 \wedge 16 / 4$$

$$3 2 \wedge 16 / 4 3$$

$$3 2 \wedge 16 / 4 3 * -$$

$$3 2 \wedge 16 / 4 3 * - 2$$

$$3 2 \wedge 16 / 4 3 * - 2 +$$

R to L (27)

R to L

converting and solving

* $+ 2 - * 24 / 16 \wedge 23$

$$+ 2 - * 24 / 16 8$$

$$+ 2 - * 24 / 16 2$$

$$+ 2 - * 24 / 16 2$$

$$+ 2 - * 24 / 16 2$$

* Prefix to infix

$$2 / 3$$

$$16 / 2 \wedge 3$$

$$16 / 2 \wedge 3$$

$$16 / 2 \wedge 3$$

$$16 / 2 \wedge 3$$

$$16 / 2 \wedge 3$$

$$24 - 16 / 2 \wedge 3$$

$$24 - 16 / 2 \wedge 3$$

$$24 - 16 / 2 \wedge 3$$

$$24 - 16 / 2 \wedge 3$$

$$24 - 16 / 2 \wedge 3$$

$$24 - 16 / 2 \wedge 3$$


```

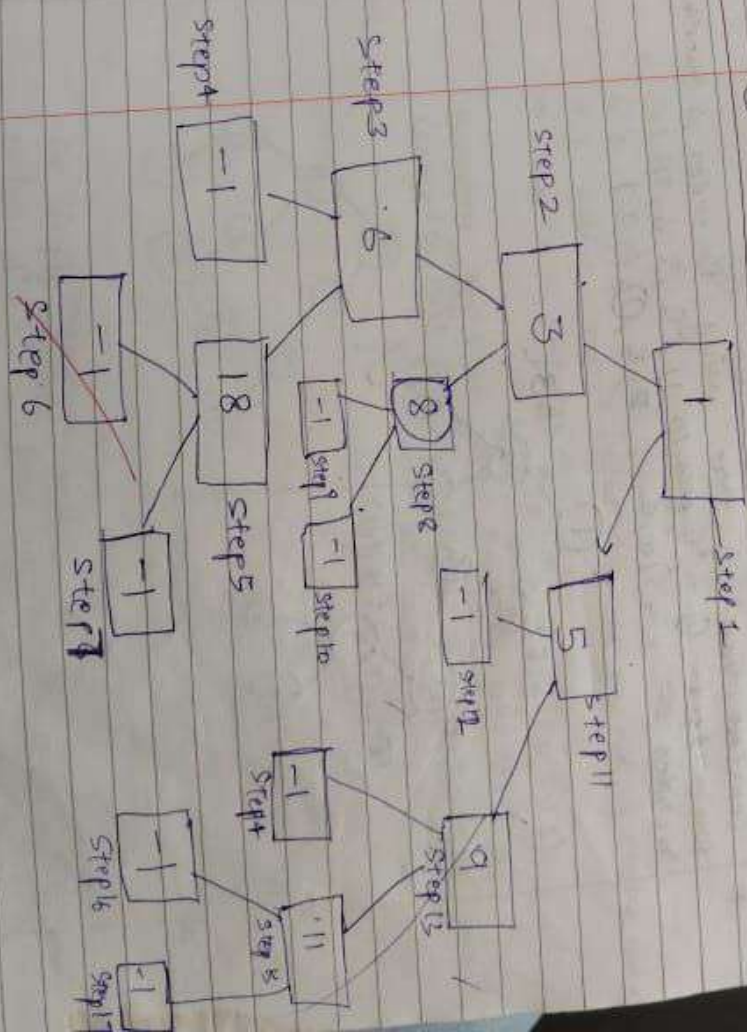
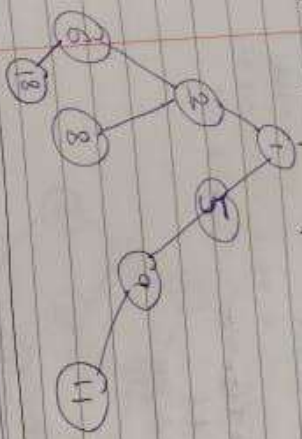
void cv
struct tree * create () {
    struct tree * tmp;
    tmp = (struct tree *) malloc (size of struct tree);
    printf ("Enter data ");
    scanf ("%d", &tmp->data);
    if (tmp->data == -1)
        return 0;
    else {
        printf ("Enter left of %d", tmp->data);
        scanf ("%d", &tmp->left);
        if (tmp->left == -1)
            tmp->left = create ();
        else
            tmp->left = create ();
        printf ("Enter right of %d", tmp->data);
        scanf ("%d", &tmp->right);
        if (tmp->right == -1)
            tmp->right = create ();
        else
            tmp->right = create ();
    }
}
    
```

```

void inorder (struct tree * root) {
    if (root == NULL)
        return 0;
    else {
        inorder (root->left);
        printf ("%d ", root->data);
        inorder (root->right);
    }
}

void preorder (struct tree * root) {
    if (root == NULL)
        return 0;
    else {
        printf ("%d ", root->data);
        preorder (root->left);
        preorder (root->right);
    }
}
    
```

Example for create Binary tree :-

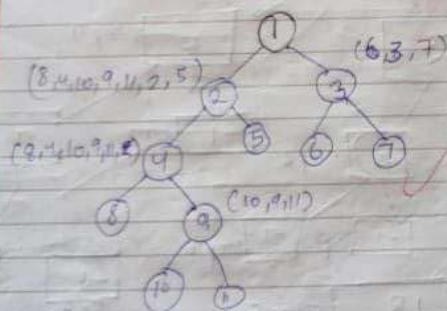


① Tree using Array :-
complete Full Binary tree is must.

If node is i th position :-
left child $\Rightarrow i \times 2 + 1$
Right child $\Rightarrow i \times 2 + 2$

Parent $\Rightarrow \frac{i-1}{2}$

* Construction of tree using Preorder & Successor
preorder :- ① 2, 4, 6, 9, 10, 11, 5, 3, 6, 7
Successor :- 8, 4, 10, 9, 11, 2, 5, ①, 6, 37



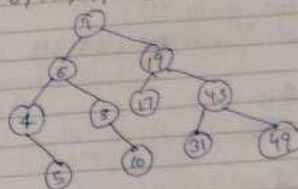
Assignment - 12

* Binary Search Tree

Insertion

$\rightarrow$ do this L to R :-

11, 6, 8, 17, 4, 10, 5, 17, 43, 49, 31



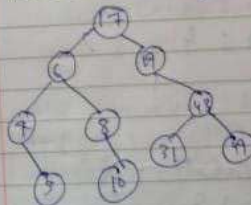
* Deletion

$\rightarrow$ 0 child $\rightarrow$ Just Remove that node

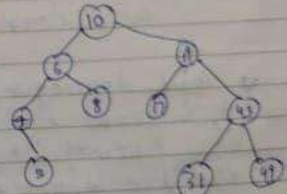
$\rightarrow$ 1 child $\rightarrow$ connect child with the parent of deleted node

$\rightarrow$ 2 child $\rightarrow$ Deleting 11

Inorder successor
Remove smallest from right



Inorder predecessor
Remove & Replace highest from left



Or simply find inorder which is ascending order of BST. And use predecessor or successor

Assignment - 15

34

→ Queue by Array :-

```
void enqueue() {
    int arr[5]; int data;
    printf("Enter data");
    scanf("%d", &data);
    if (rear == 4) {
        int rear = 0; -1;
        int front = 0; -1;
    }
```

```
if (rear == 4) {
    printf("Overflow"); }
else if (rear == -1) {
    rear++;
    front++;
    arr[rear] = data;
}
```

```
else {
    rear++;
    arr[rear] = data; }
}
```

```
void dequeue() {
    if (front == -1) {
        printf("Underflow"); }
    else if (front == rear) {
        front = rear = -1;
    }
```

```
else {
    front++; }
}
```

```
void display() {
    for (int i = front; i <= rear; i++) {
    }
```

35

```
printf("%d", arr[i]); }
```

```
if (front == -1) {
    printf("No value"); }
```

* Circular Queue by Array :-

```
void enqueue() {
    int data;
    printf("Enter data : ");
    scanf("%d", &data);
    if ((rear + 1) % 5 == front) {
        printf("Overflow"); }
    else if (front == -1) {
        rear++;
        front++;
        queue[rear] = data; }
```

```
else {
    rear = (rear + 1) % 5;
    queue[rear] = data;
}
```

```
void dequeue() {
    if (front == -1) {
        printf("Underflow"); }
    else if (front == rear) {
        front = rear = -1;
    }
```

```
else {
    front = (front + 1) % 5;
}
```



```
void display () {  
    if (front == -1) {  
        printf ("Empty Queue");  
    }  
    else {  
        int i = front;  
        while (i != rear) {  
            printf ("%d \t", queue[i]);  
            i = (i + 1) % 5;  
        }  
        printf ("%d \t", queue[rear]);  
    }  
}
```

on my